

Laboratorio 2 - Terraform y Docker

Datos generales

- **Curso:** Fundamentos de Programación en la Nube
- **Estudiante:** Oscar Marín Zamora
- **Tema:** Infraestructura como código con Terraform y Docker
- **Fecha:** 3 de junio de 2026

Objetivo

Implementar una infraestructura local con contenedores Docker utilizando Terraform como herramienta de infraestructura como código. Durante el laboratorio se creó una red personalizada, se descargó la imagen oficial de Nginx, se desplegaron tres contenedores, se verificó el acceso desde el navegador, se probó la comunicación interna entre servicios y se revisó cómo Terraform detecta cambios entre lo que existe actualmente y lo que está definido en la configuración.

La infraestructura como código, también conocida como IaC, consiste en definir recursos de infraestructura mediante archivos de configuración. En lugar de crear cada recurso de forma manual, se describe lo que se necesita en un archivo, en este caso `main.tf`, y Terraform se encarga de aplicar esa configuración. Esta forma de trabajo permite repetir despliegues, documentar mejor los cambios y mantener la infraestructura junto con el resto del proyecto.

Parte 1 - Verificación del entorno

Primero se verificó que Docker y Terraform estuvieran instalados y funcionando correctamente en el equipo. Esta revisión era necesaria porque Terraform utiliza Docker para crear la red, descargar la imagen y administrar los contenedores del laboratorio.

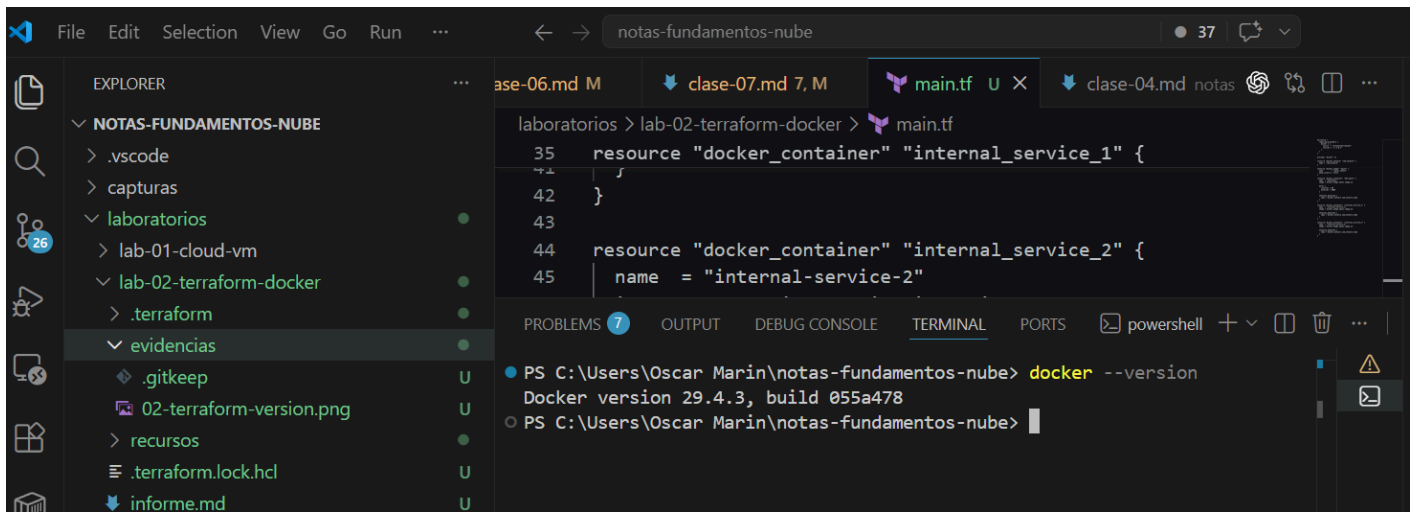
Comandos utilizados:

```
docker --version
```

```
terraform version
```

```
docker ps
```

Evidencias:

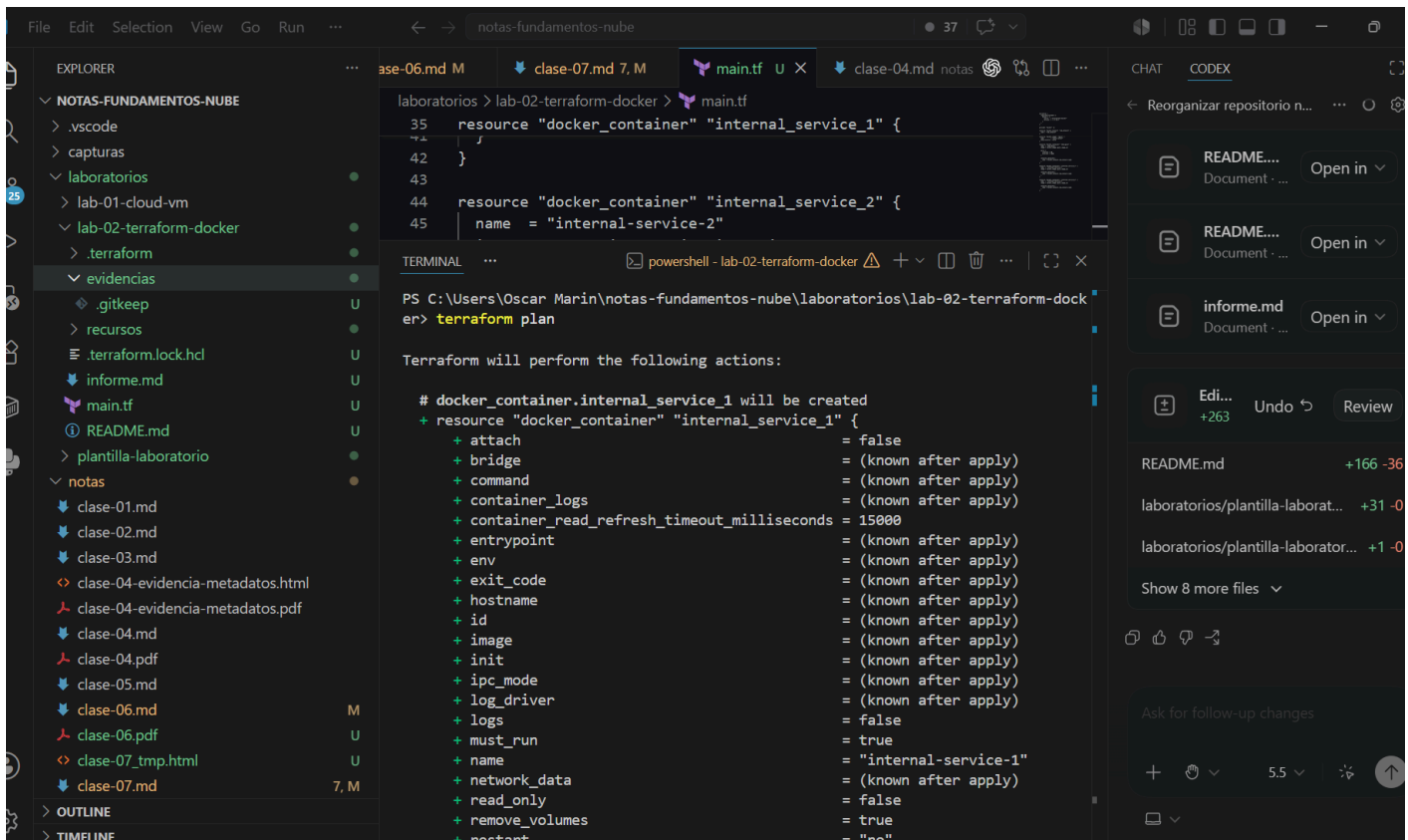


The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays the project structure for 'NOTAS-FUNDAMENTOS-NUBE', including folders like 'laboratorios' and 'evidencias'. The main editor window shows a Terraform configuration file 'main.tf' with the following content:

```
laboratorios > lab-02-terraform-docker > main.tf
35 resource "docker_container" "internal_service_1" {
42 }
43
44 resource "docker_container" "internal_service_2" {
45   name = "internal-service-2"
}
```

The TERMINAL panel at the bottom shows the output of the 'docker --version' command:

```
PS C:\Users\Oscar Marin\notas-fundamentos-nube> docker --version
Docker version 29.4.3, build 055a478
PS C:\Users\Oscar Marin\notas-fundamentos-nube>
```



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays the project structure for 'NOTAS-FUNDAMENTOS-NUBE', including folders like 'laboratorios' and 'evidencias'. The main editor window shows a Terraform configuration file 'main.tf' with the following content:

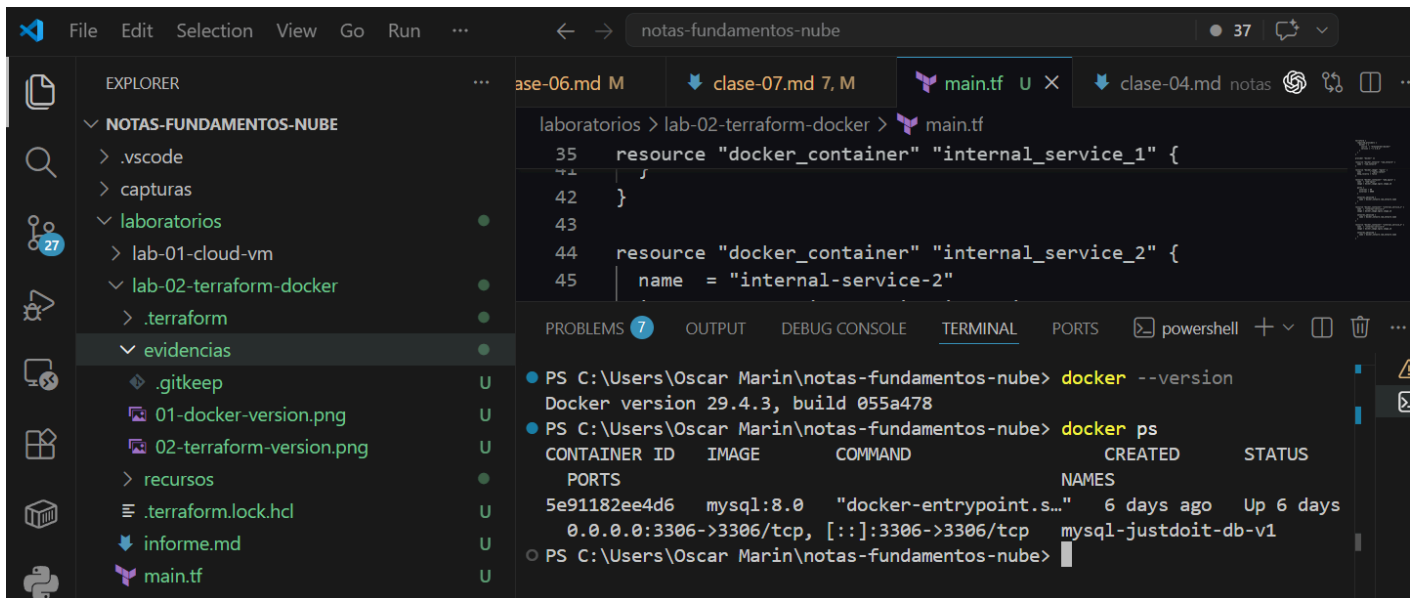
```
laboratorios > lab-02-terraform-docker > main.tf
35 resource "docker_container" "internal_service_1" {
42 }
43
44 resource "docker_container" "internal_service_2" {
45   name = "internal-service-2"
}
```

The TERMINAL panel at the bottom shows the output of the 'terraform plan' command:

```
PS C:\Users\Oscar Marin\notas-fundamentos-nube> terraform plan

Terraform will perform the following actions:

# docker_container.internal_service_1 will be created
+ resource "docker_container" "internal_service_1" {
+   attach                = false
+   bridge                = (known after apply)
+   command               = (known after apply)
+   container_logs        = (known after apply)
+   container_read_refresh_timeout_milliseconds = 15000
+   entrypoint            = (known after apply)
+   env                   = (known after apply)
+   exit_code              = (known after apply)
+   hostname               = (known after apply)
+   id                     = (known after apply)
+   image                  = (known after apply)
+   init                   = (known after apply)
+   ipc_mode               = (known after apply)
+   log_driver             = (known after apply)
+   logs                   = false
+   must_run               = true
+   name                   = "internal-service-1"
+   network_data           = (known after apply)
+   read_only              = false
+   remove_volumes        = true
+   restart                = "no"
}
```



Resultado:

- Docker v29.4.3 se encontró instalado y operativo.
- Terraform v1.15.4 se encontró instalado y operativo.
- Se verificó el estado inicial de los contenedores antes del despliegue.

Parte 2 - Creación del proyecto

Se creó el proyecto del laboratorio dentro de `laboratorios/lab-02-terraform-docker/`. El archivo principal fue `main.tf`, donde se definieron el proveedor de Docker, la red personalizada, la imagen oficial de Nginx y los tres contenedores utilizados en la práctica.

Comandos utilizados para ubicarse en el proyecto:

```
cd laboratorios/lab-02-terraform-docker
```

Comando utilizado para revisar el archivo principal del laboratorio:

```
Get-Content main.tf
```

En Terraform, un proveedor, o Provider, es el componente que permite trabajar con una plataforma específica. En este laboratorio se utilizó el Provider `kreuzwerker/docker`, que permite que Terraform se comuniquen con Docker. Gracias a este componente, Terraform puede crear redes, descargar imágenes y administrar contenedores a partir de la configuración escrita en `main.tf`.

Configuración utilizada en `main.tf`:

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0.2"
    }
  }
}

provider "docker" {}

resource "docker_network" "lab_network" {
  name = "lab_network"
}

resource "docker_image" "nginx" {
  name          = "nginx:latest"
  keep_locally = false
}

resource "docker_container" "web_nginx" {
  name     = "web-nginx"
  image    = docker_image.nginx.image_id

  ports {
    internal = 80
    external = 9090
  }

  networks_advanced {
    name = docker_network.lab_network.name
  }
}

resource "docker_container" "internal_service_1" {
  name     = "internal-service-1"
  image    = docker_image.nginx.image_id

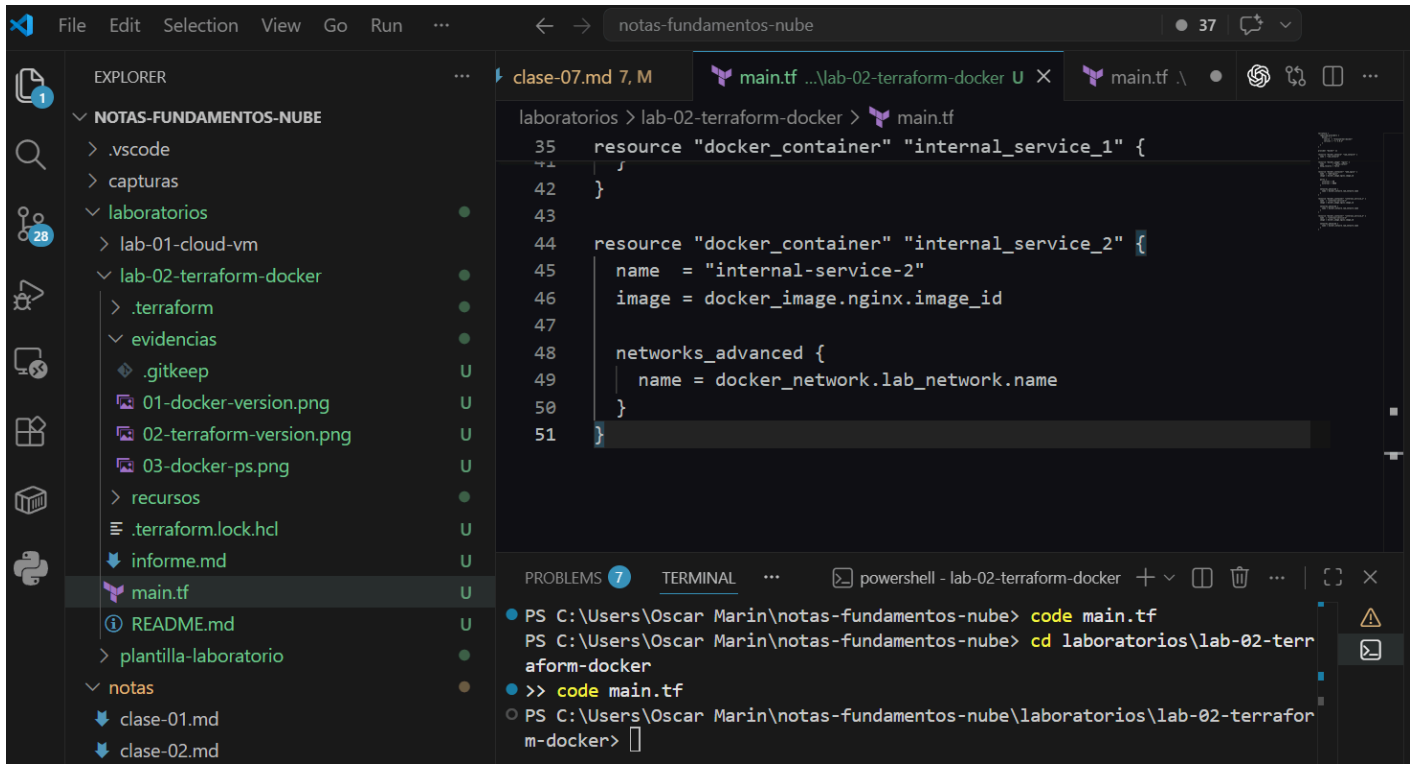
  networks_advanced {
    name = docker_network.lab_network.name
  }
}

resource "docker_container" "internal_service_2" {
  name     = "internal-service-2"
  image    = docker_image.nginx.image_id

  networks_advanced {
    name = docker_network.lab_network.name
  }
}
```

```
}  
}
```

Evidencia:



La red personalizada `lab_network` se usó para mantener los contenedores del laboratorio dentro de una misma red Docker. Esto permite que los servicios se comuniquen entre sí usando sus nombres de contenedor, sin depender de direcciones IP fijas.

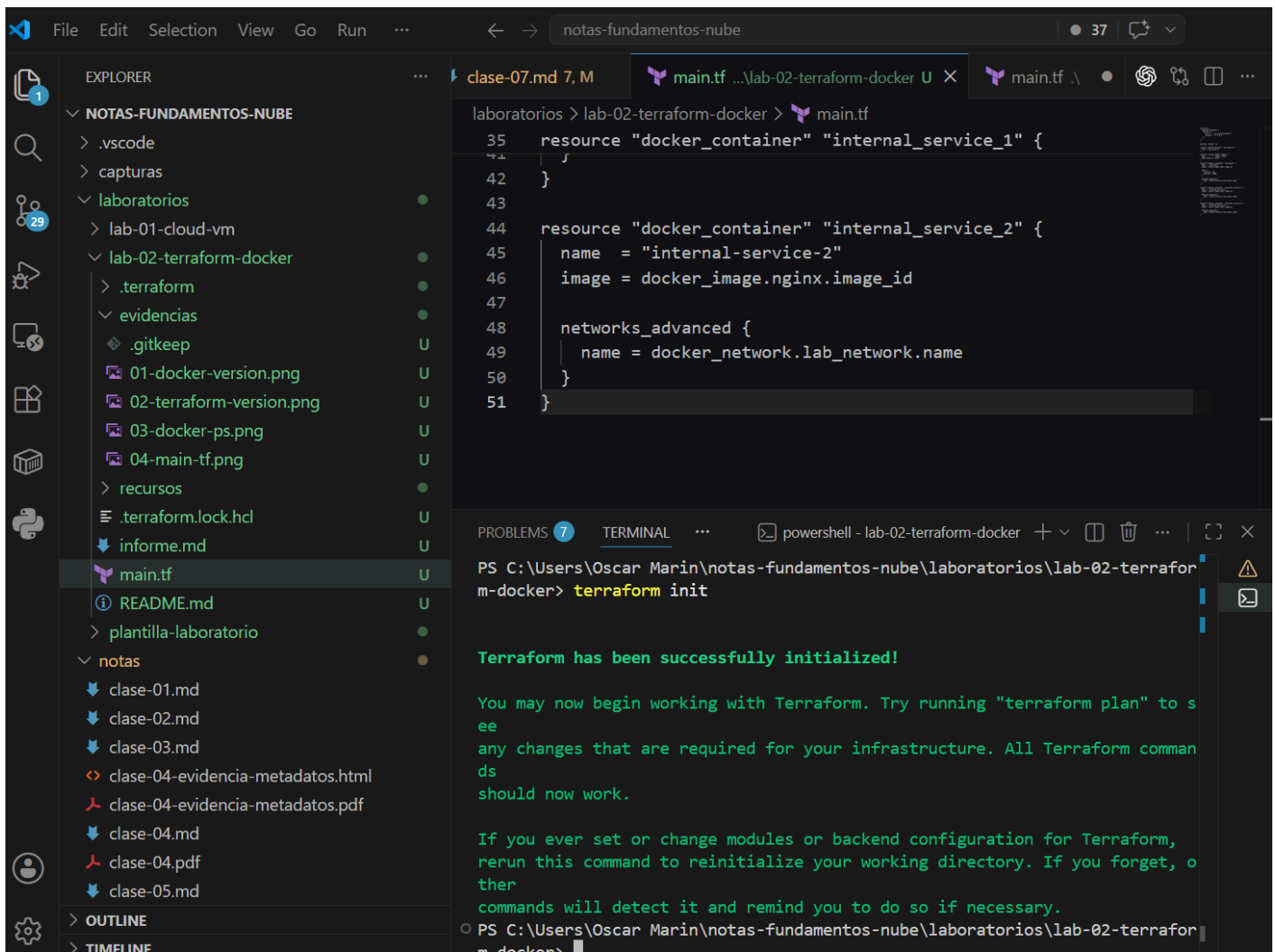
Parte 3 - Inicialización y despliegue

Después de crear `main.tf`, se inicializó el proyecto con Terraform. Al ejecutar este comando, Terraform descargó el Provider necesario y preparó el directorio de trabajo.

Comando utilizado:

```
terraform init
```

Evidencia:



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays a project structure for 'NOTAS-FUNDAMENTOS-NUBE'. The file 'main.tf' is selected under the 'lab-02-terraform-docker' directory. The main editor shows the Terraform configuration for two internal services. The terminal at the bottom shows the command 'terraform init' being executed successfully, with a confirmation message and instructions on how to use Terraform.

```
laboratorios > lab-02-terraform-docker > main.tf
35 resource "docker_container" "internal_service_1" {
42 }
43
44 resource "docker_container" "internal_service_2" {
45   name = "internal-service-2"
46   image = docker_image.nginx.image_id
47
48   networks_advanced {
49     name = docker_network.lab_network.name
50   }
51 }
```

```
PS C:\Users\Oscar Marin\notas-fundamentos-nube\laboratorios\lab-02-terraform-docker> terraform init

Terraform has been successfully initialized!

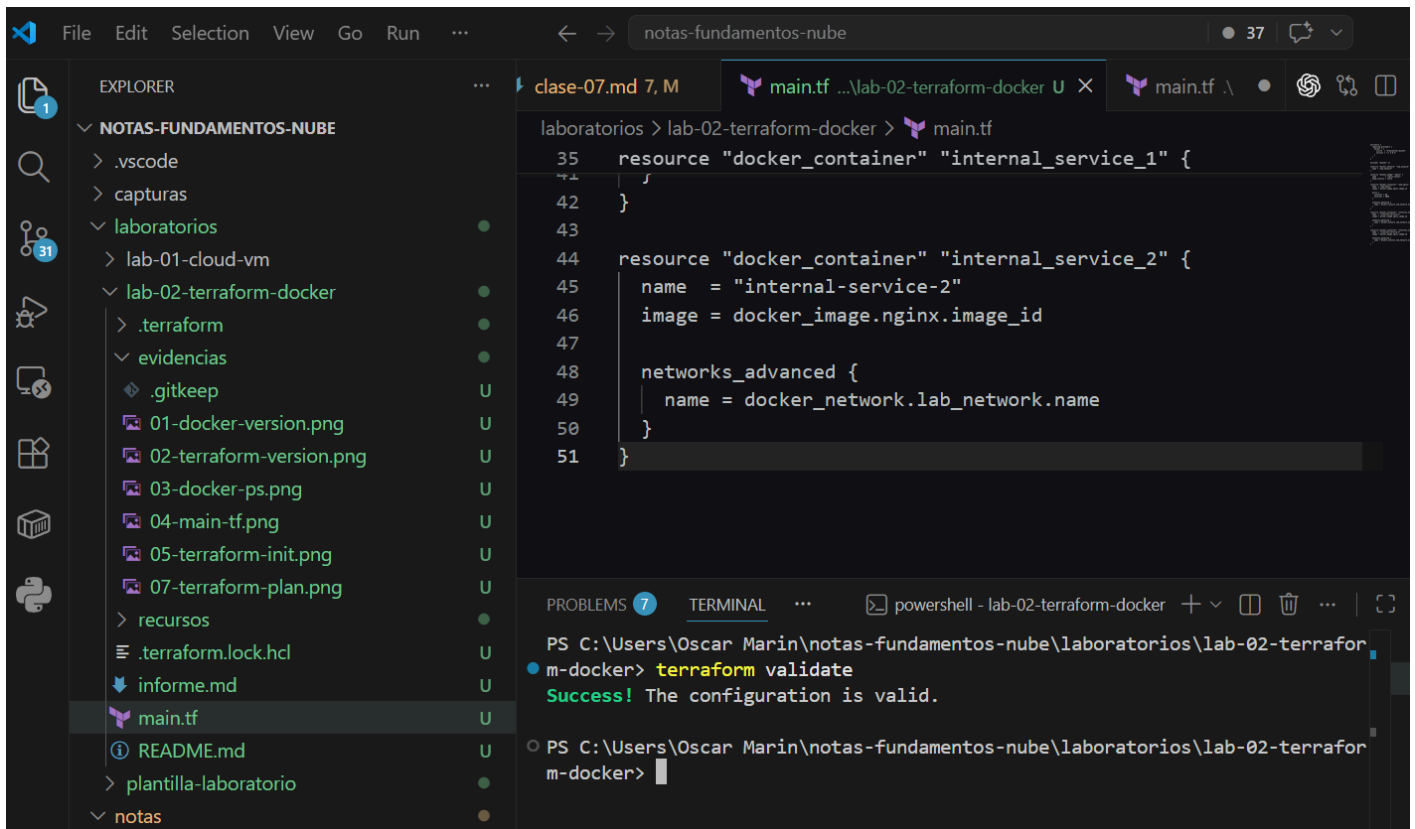
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Luego se validó la sintaxis de la configuración:

```
terraform validate
```

Evidencia:



Posteriormente se revisó el plan de ejecución antes de crear los recursos:

```
terraform plan
```

Evidencia:

The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left displaying the project structure. The main editor shows the file `clase-07.md` at line 7, which contains Terraform code for two Docker containers. The terminal at the bottom shows the execution of `terraform plan` in a PowerShell session, displaying the plan for adding resources and the resulting plan summary.

```
laboratorios > lab-02-terraform-docker > main.tf
35 resource "docker_container" "internal_service_1" {
42 }
43
44 resource "docker_container" "internal_service_2" {
45   name = "internal-service-2"
46   image = docker_image.nginx.image_id
47
48   networks_advanced {
49     name = docker_network.lab_network.name
50   }
51 }
```

```
PS C:\Users\Oscar Marin\notas-fundamentos-nube\laboratorios\lab-02-terrafor
m-docker> terraform plan
+ ipam_driver = "default"
+ name       = "lab_network"
+ options    = (known after apply)
+ scope      = (known after apply)

+ ipam_config (known after apply)
}

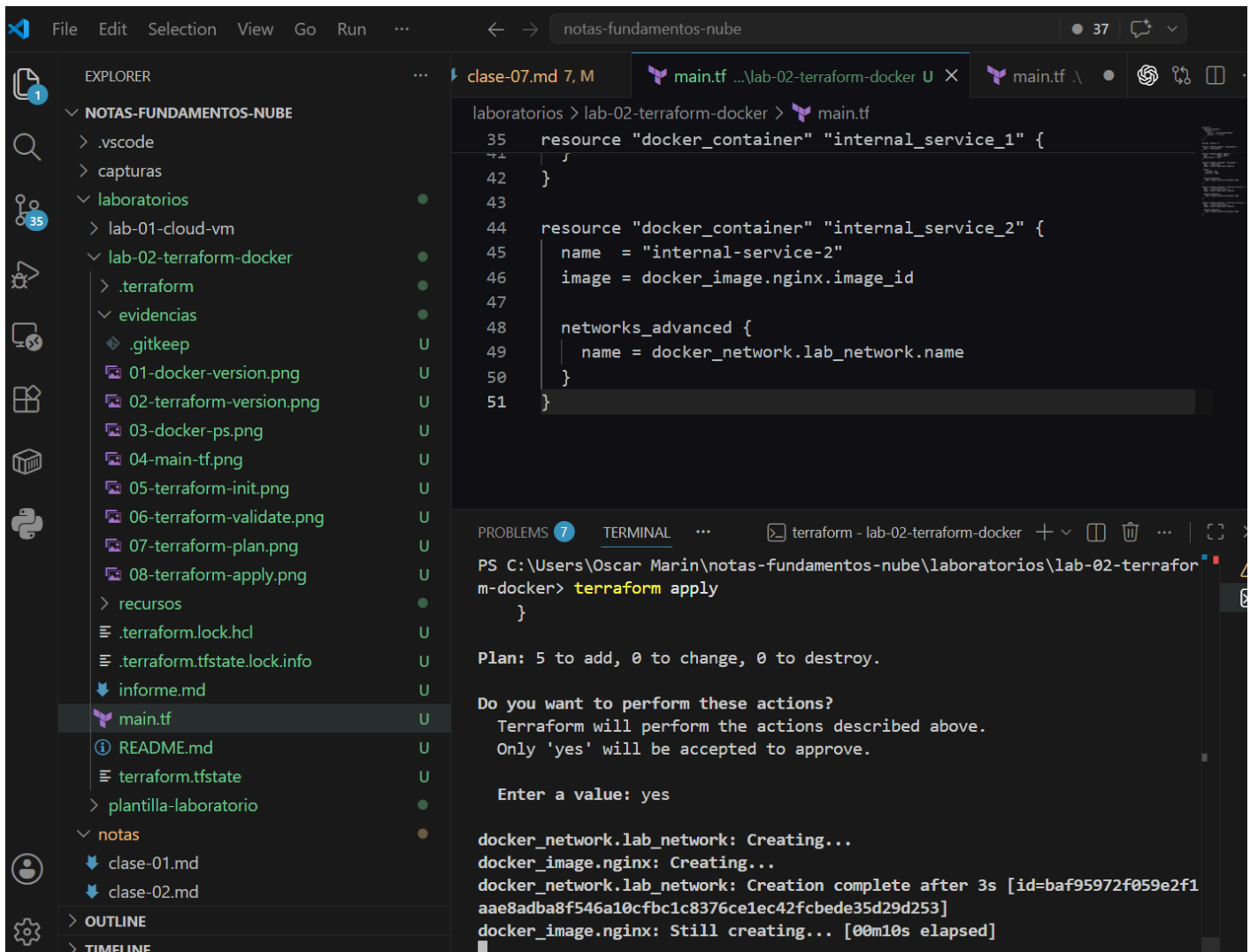
Plan: 5 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\Users\Oscar Marin\notas-fundamentos-nube\laboratorios\lab-02-terrafor
m-docker>
```

Finalmente se aplicó la configuración:

```
terraform apply
```

Evidencia:



The screenshot shows the Visual Studio Code interface with the Explorer view on the left, the main editor in the center, and the Terminal view at the bottom. The Explorer view shows a project structure for 'NOTAS-FUNDAMENTOS-NUBE' with a subdirectory 'laboratorios' containing 'lab-02-terraform-docker'. The main editor displays the Terraform configuration file 'main.tf' for 'lab-02-terraform-docker', which defines two 'docker_container' resources: 'internal_service_1' and 'internal_service_2'. The terminal shows the command 'terraform apply' being executed, resulting in the creation of the 'lab_network' and the 'nginx' image, with the status 'Still creating... [00m10s elapsed]'.

```
laboratorios > lab-02-terraform-docker > main.tf
35 resource "docker_container" "internal_service_1" {
42 }
43
44 resource "docker_container" "internal_service_2" {
45   name = "internal-service-2"
46   image = docker_image.nginx.image_id
47
48   networks_advanced {
49     name = docker_network.lab_network.name
50   }
51 }
```

```
PS C:\Users\Oscar Marin\notas-fundamentos-nube\laboratorios\lab-02-terraform-docker> terraform apply
}

Plan: 5 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

docker_network.lab_network: Creating...
docker_image.nginx: Creating...
docker_network.lab_network: Creation complete after 3s [id=baf95972f059e2f1
aae8adba8f546a10cfbc1c8376ce1ec42fcbde35d29d253]
docker_image.nginx: Still creating... [00m10s elapsed]
```

Como resultado, Terraform creó la red `lab_network`, descargó la imagen oficial `nginx:latest` y creó los contenedores `web-nginx`, `internal-service-1` e `internal-service-2`.

El archivo `terraform.tfstate` es el archivo donde Terraform guarda el estado de los recursos que administra. En este laboratorio registra los recursos creados, sus atributos principales y la relación entre lo definido en `main.tf` y lo que existe realmente en Docker. Este archivo es importante porque Terraform lo utiliza para saber qué recursos ya existen y qué cambios debe aplicar en futuras ejecuciones.

Parte 4 - Verificación del despliegue

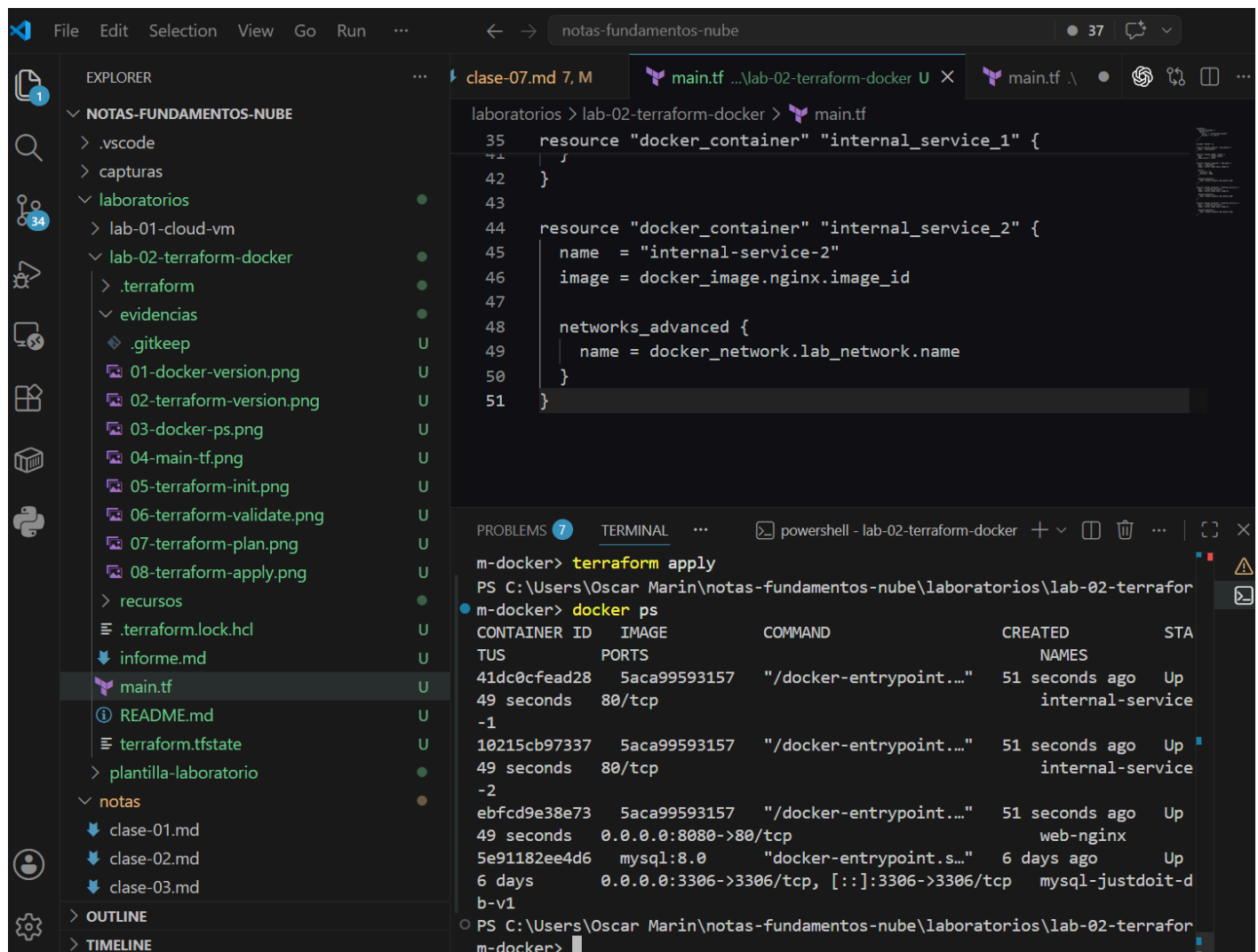
Una vez aplicado el despliegue, se verificó que los contenedores estuvieran en ejecución.

Comandos utilizados:

```
docker ps
```

```
docker ps -a
```

Evidencia:



The screenshot shows the VS Code interface with the Explorer view on the left, the main editor in the center, and the Terminal view at the bottom.

Explorer View: The file tree shows the project structure under "NOTAS-FUNDAMENTOS-NUBE". The "lab-02-terraform-docker" folder is expanded, showing subfolders like ".terraform", "evidencias", and "recursos". The file "main.tf" is selected.

Main Editor: The file "main.tf" is open, showing Terraform configuration for two Docker containers. The configuration includes resource definitions for "internal_service_1" and "internal_service_2", with network settings for "lab_network".

```
35 resource "docker_container" "internal_service_1" {
36     name = "internal-service-1"
37     image = docker_image.nginx.image_id
38     networks_advanced {
39         name = docker_network.lab_network.name
40     }
41 }
42
43
44 resource "docker_container" "internal_service_2" {
45     name = "internal-service-2"
46     image = docker_image.nginx.image_id
47     networks_advanced {
48         name = docker_network.lab_network.name
49     }
50 }
51 }
```

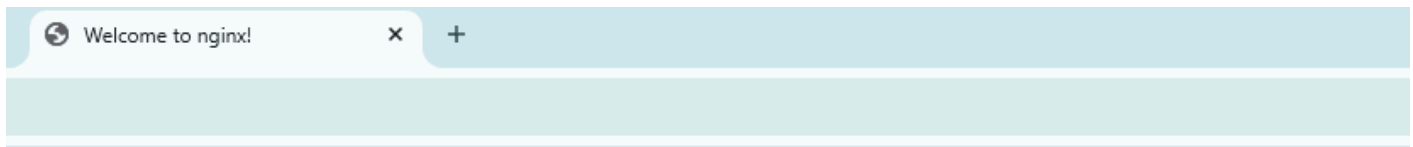
Terminal View: The terminal shows the execution of Terraform commands. The "terraform apply" command was run, and the output shows the creation of two containers: "internal-service-1" and "internal-service-2". The "docker ps" command was also run, showing the status of the containers.

```
m-docker> terraform apply
PS C:\Users\Oscar Marin\notas-fundamentos-nube\laboratorios\lab-02-terrafor
m-docker> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
41dc0cfad28    5aca99593157  "/docker-entrypoint..."  51 seconds ago Up            80/tcp
10215cb97337    5aca99593157  "/docker-entrypoint..."  51 seconds ago Up            80/tcp
ebfcd9e38e73    5aca99593157  "/docker-entrypoint..."  51 seconds ago Up            0.0.0.0:8080->80/tcp
5e91182ee4d6    mysql:8.0      "docker-entrypoint.s..."  6 days ago    Up            0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
```

También se verificó el acceso al servidor Nginx desde el navegador. Inicialmente el contenedor **web-nginx** fue expuesto usando el puerto externo **8080**, por lo que el acceso se realizó desde:

```
http://localhost:8080
```

Evidencia:



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Esta prueba permitió confirmar que el puerto publicado en el equipo local redirigía correctamente hacia el puerto interno 80 del contenedor.

Parte 5 - Comunicación interna entre contenedores

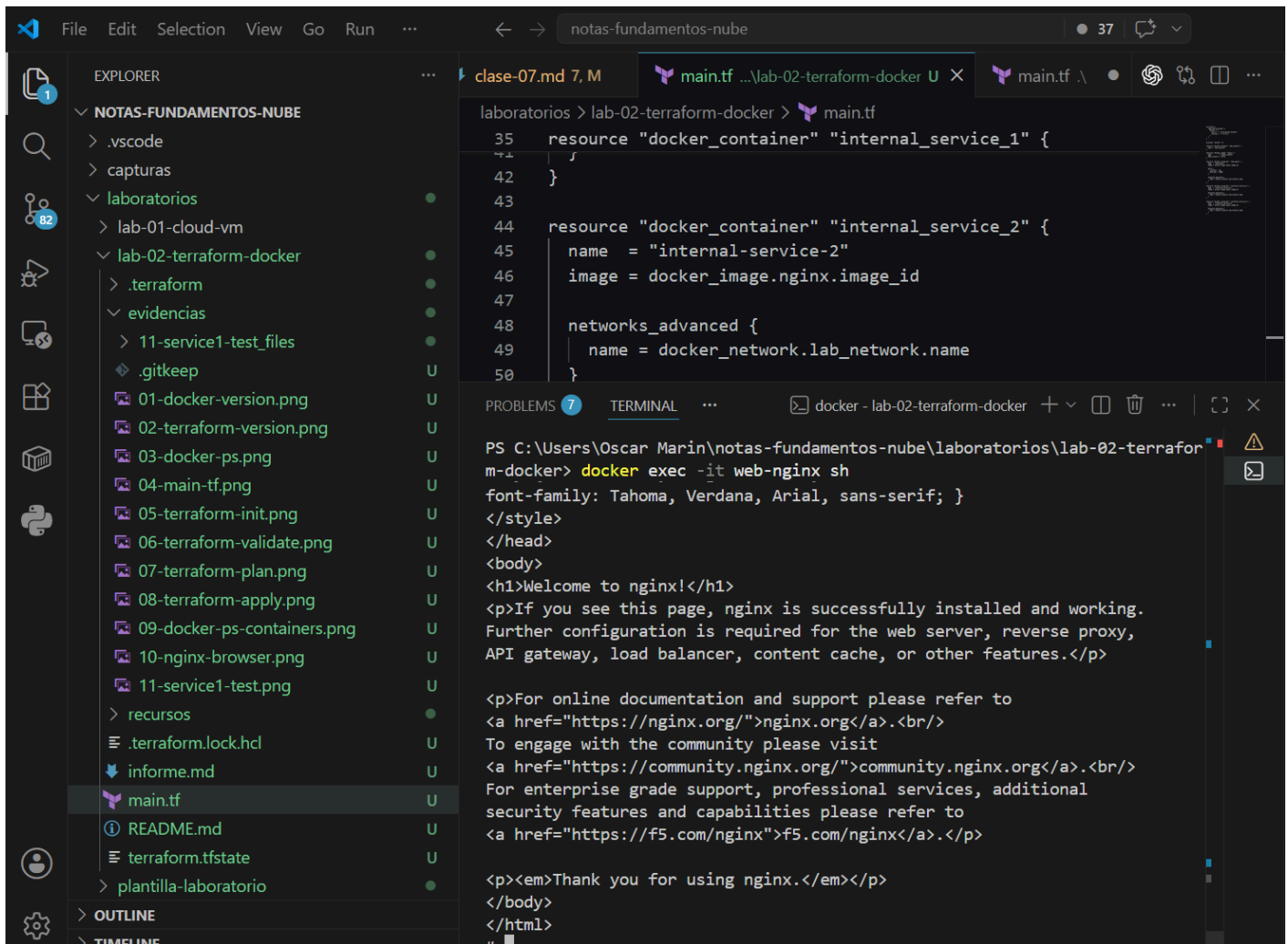
En esta etapa se verificó que los contenedores pudieran comunicarse dentro de `lab_network`. Para la prueba se ejecutó `curl` desde un contenedor hacia otros servicios, usando el nombre del contenedor como dirección.

Comandos utilizados:

```
docker exec -it internal-service-1 curl http://web-nginx
```

```
docker exec -it internal-service-1 curl http://internal-service-2
```

Evidencia:



Los contenedores pueden comunicarse sin utilizar IP porque Docker incluye resolución interna de nombres en las redes personalizadas. Cuando varios contenedores están conectados a la misma red, cada nombre de contenedor puede funcionar como un nombre de host. Esto es útil porque las direcciones IP internas pueden cambiar si un contenedor se elimina y se vuelve a crear.

Docker resuelve estos nombres mediante su DNS interno. En una red personalizada, Docker mantiene el registro de los contenedores conectados y permite que una solicitud a `web-nginx` se dirija hacia la dirección interna actual de ese contenedor. Por eso `internal-service-1` pudo comunicarse con `web-nginx` e `internal-service-2` usando nombres, sin conocer sus direcciones IP.

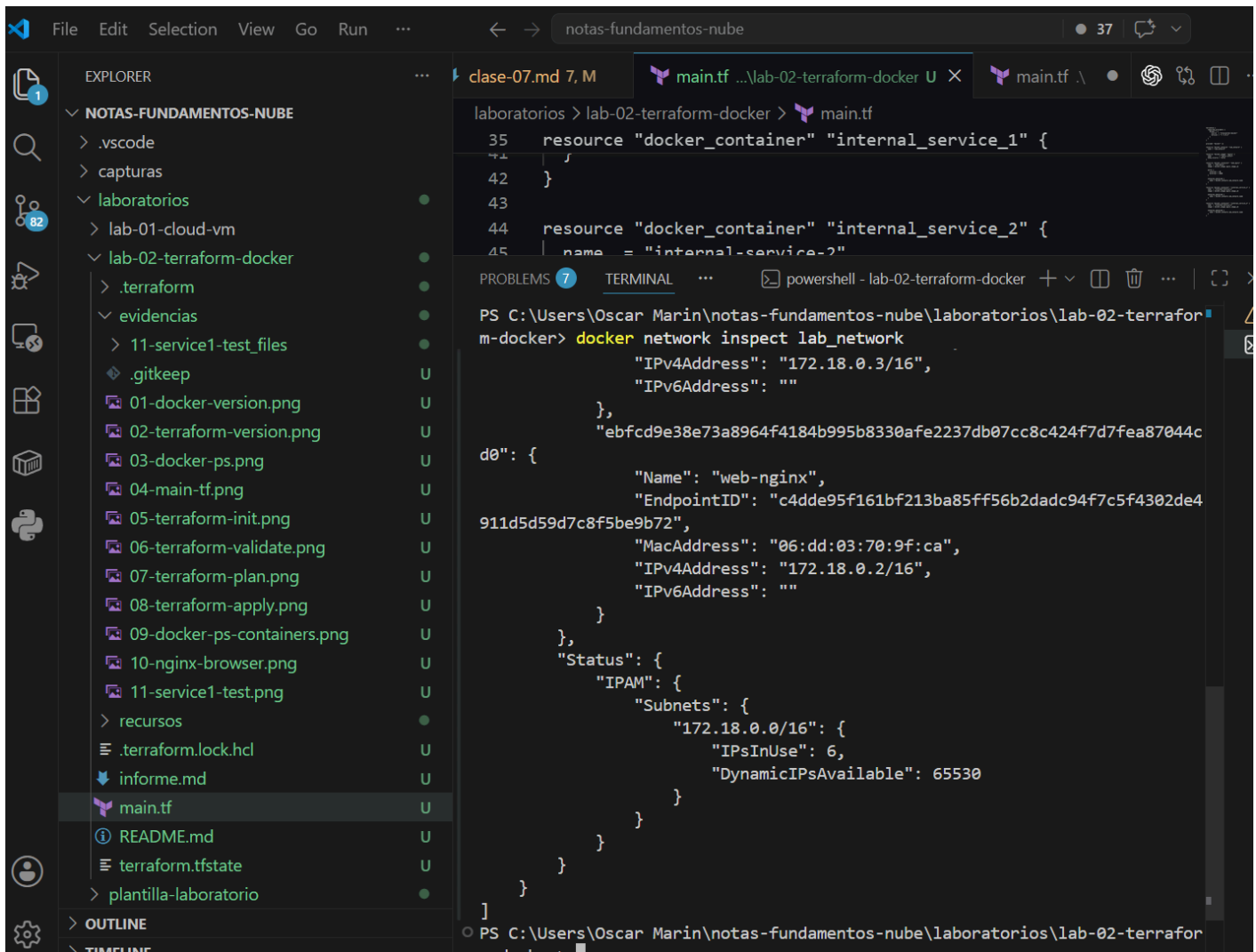
Parte 6 - Inspección de red

Se inspeccionó la red `lab_network` para revisar su configuración y confirmar qué contenedores estaban conectados.

Comando utilizado:

```
docker network inspect lab_network
```

Evidencia:



El comando `docker network inspect` muestra información detallada de una red Docker. Entre los datos más importantes se encuentran el identificador de la red, el controlador utilizado, el alcance, la subred, la puerta de enlace, las opciones de configuración y la lista de contenedores conectados con sus direcciones IP internas.

Esta información permitió validar que los tres contenedores del laboratorio pertenecían a la misma red y tenían conectividad interna. También sirve para diagnosticar problemas de comunicación y confirmar que los recursos creados por Terraform existen en Docker.

Parte 7 - Realización de cambios

Posteriormente se modificó el puerto externo del contenedor `web-nginx`. Al inicio el puerto publicado era `8080`, pero luego se cambió a `9090` dentro del bloque `ports` del recurso `docker_container.web_nginx`.

Cambio aplicado en `main.tf`:

```
ports {
  internal = 80
  external = 9090
}
```

Comando utilizado para revisar el cambio:

```
terraform plan
```

Evidencia:

```
laboratorios > lab-02-terraform-docker > main.tf
19 }
20
21 resource "docker_container" "web_nginx" {
22   name = "web-nginx"
23   image = docker_image.nginx.image_id
24
25   ports {
26     internal = 80
27     external = 9090
28   }
29 }
```

```
PS C:\Users\Oscar Marin\notas-fundamentos-nube\laboratorios\lab-02-terraform-docker> terraform plan
~ healthcheck (known after apply)
~ labels (known after apply)
~ ports {
  ~ external = 8080 -> 9090 # forces replacement
    # (3 unchanged attributes hidden)
}
# (1 unchanged block hidden)
}
Plan: 3 to add, 0 to change, 3 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.
```

Al ejecutar `terraform plan`, Terraform detectó que la configuración deseada en `main.tf` ya no coincidía con el estado registrado en `terraform.tfstate` ni con la configuración previa del contenedor. Esto ocurrió porque el puerto externo cambió de `8080` a `9090`.

En Terraform, el estado deseado es lo que está definido en los archivos `.tf`. El estado actual corresponde a los recursos que existen en la infraestructura y al registro que Terraform mantiene en `terraform.tfstate`. Cuando hay diferencias entre ambos, Terraform calcula qué acciones necesita realizar para que la infraestructura vuelva a coincidir con la configuración definida.

En este caso, el cambio del puerto externo obligó a recrear el recurso. La publicación de puertos de un contenedor Docker no se modifica de forma directa sobre un contenedor ya creado; normalmente se debe crear un contenedor nuevo con la nueva configuración. Por esta razón, Terraform marcó el recurso para reemplazo en lugar de aplicar solo una actualización simple.

Comando utilizado para aplicar el cambio:

```
terraform apply
```

Después de aplicar el cambio, el acceso esperado al servicio Nginx queda en:

```
http://localhost:9090
```

Parte 8 - Destrucción de infraestructura

Al finalizar el laboratorio se debe destruir la infraestructura creada para liberar recursos locales y comprobar que Terraform puede administrar también la eliminación de los recursos.

Comando pendiente:

```
terraform destroy
```

Verificación pendiente posterior a la destrucción:

```
docker ps -a
```

```
docker network ls
```

Marcadores de evidencias pendientes:

 Destruccion de infraestructura pendiente

 Verificacion final pendiente

Problemas encontrados y solución aplicada

Problema	Causa probable	Solución aplicada
Era necesario comprobar que Terraform y Docker estuvieran disponibles antes del despliegue.	Terraform necesita comunicarse con Docker para administrar los recursos definidos en <code>main.tf</code> .	Se ejecutaron <code>docker --version</code> , <code>terraform version</code> y <code>docker ps</code> antes de iniciar el despliegue.
Los contenedores internos no tenían puertos publicados	Solo <code>web-nginx</code> necesitaba acceso desde el navegador; los	Se validaron mediante comunicación interna usando <code>docker exec</code> y <code>curl</code>

Problema	Causa probable	Solución aplicada
hacia el equipo local.	otros servicios se usaron para pruebas internas.	dentro de la red personalizada.
El cambio del puerto externo no se aplicaba como una modificación directa.	La configuración de puertos de un contenedor Docker requiere recrear el contenedor.	Se revisó <code>terraform plan</code> para entender el reemplazo y luego se aplicó el cambio con <code>terraform apply</code> .
Quedan evidencias finales pendientes.	La destrucción de infraestructura y la verificación final todavía no se han documentado con capturas.	Se dejaron marcadores en el informe para agregar <code>14-terraform-destroy.png</code> y <code>15-verificacion-final.png</code> .

Conclusiones

En este laboratorio comprendí cómo Terraform permite administrar infraestructura local de Docker mediante archivos de configuración. Al definir la red, la imagen y los contenedores en `main.tf`, pude representar la infraestructura como código y repetir el despliegue de una forma más ordenada que si hubiera ejecutado todos los comandos manualmente.

También observé la importancia del Provider, ya que funciona como el enlace entre Terraform y Docker. Sin este componente, Terraform no podría interpretar cómo crear contenedores, imágenes o redes dentro del entorno local.

Aprendí que el archivo `terraform.tfstate` es fundamental para que Terraform conozca los recursos que administra. Este archivo permite comparar el estado actual con el estado deseado y detectar cambios, como la modificación del puerto externo de `8080` a `9090`.

Además, comprobé que una red personalizada de Docker facilita la comunicación interna entre contenedores. En lugar de depender de direcciones IP, los contenedores pueden comunicarse usando nombres como `web-nginx` e `internal-service-2`, lo cual hace que la configuración sea más estable y fácil de mantener.

Finalmente, concluyo que Terraform y Docker se complementan bien para practicar conceptos de infraestructura en la nube. Aunque el laboratorio se realizó de forma local, los principios de infraestructura como código, estado deseado, despliegue reproducible y administración de cambios también se aplican en entornos reales de nube.